

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ**

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

**«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)**

Факультет компьютерных технологий и прикладной математики

Кафедра информационных технологий

ОТЧЕТ

**о выполнении лабораторной работы № 6
по дисциплине «Программная инженерия»**

Выполнила: ст. гр. МО 32/2

Переузник В. С.

Проверил: доцент каф. ИТ

Полетайкин А. Н.

Краснодар

2026

Тема: комплексное тестирование ПС

Цель: закрепление методов и способов тестирования ПО и освоение методики комплексного тестирования разработанной программной системы методом белого ящика.

Задание: основываясь на результатах внешнего проектирования ПС, полученных при выполнении лабораторной работы 1, выполнить комплексное тестирование работоспособности ПС. Для этого необходимо:

1. Выполнить интеграционное тестирование на компонентном уровне разработанных автоматизированных функций ПС, в том числе тестирование взаимодействия компонентов ПС с базой данных, используя подход Bottom Up Integration.
2. Согласно функциональным требованиям к ПС, разработать несколько сценариев тестирования ПС. Реализовать сценарии в отношении разработанной ПС. Оценить факторы, оказывающие существенное влияние на эффективность нормального функционирования ПС.
3. Выполнить нагрузочное тестирование ПС и оценить эффективность её автоматизированных функций. В качестве критерия эффективности использовать время выполнения функции. Построить графики зависимости времени выполнения функций от приложенной нагрузки. Оценить факторы, оказывающие существенное влияние на эффективность функционирования ПС под нагрузкой.
4. Выполнить системное тестирование ПС методом requirements based для проверки нефункциональных требований.
5. Выполнить тестирование взаимодействия ПС с внешним окружением. Предварительно идентифицировать и описать внешние сущности, основываясь на внешнем дополнении диаграммы «Черный ящик». Оценить факторы, оказывающие существенное влияние на эффективность взаимодействия.

Все обнаруженные несоответствия задокументировать в виде баг-репортов.

Ход выполнения работы:

1. Интеграционное тестирование

Было проведено интеграционное тестирование компонентного уровня разработанных автоматизированных функций ПС с использованием

подхода Bottom Up Integration (снизу вверх). Тестирование включало проверку взаимодействия следующих компонентов:

- 1) Парсер математических выражений (math-parser.js) + вычислительный модуль (newton.js). Проверка преобразования строки в функцию и вычисления корня уравнения.
- 2) Вычислительный модуль + серверное API (app.py). Проверка передачи результата решения через эндпоинт /api/save-result.
- 3) Серверное API + СУБД PostgreSQL. Проверка сохранения данных в таблицу training_data.
- 4) Пользовательский интерфейс + вычислитель + БД. Проверка полного цикла: ввод уравнения через интерфейс, вычисление, сохранение результата.

Полученные результаты:

Тест 1. Парсер + вычислительный модуль

- Введено уравнение $x^2 - 4 = 0$, выбран метод Ньютона
- Парсер преобразовал выражение в $\text{pow}(x,2)-4$ (видно в консоли браузера на рисунке 2)
- Метод Ньютона нашел корень $x = 2.000000$ за 5 итераций
- Невязка составила 0.00000

В результате видим, что компоненты работают корректно.

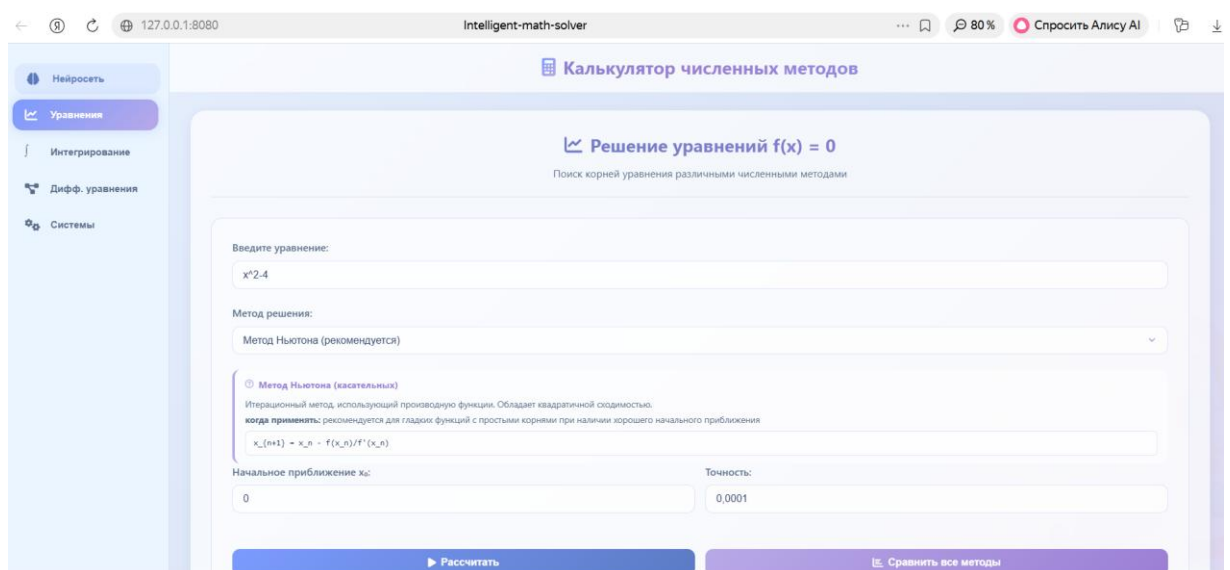


Рисунок 1. Ввод тестового примера

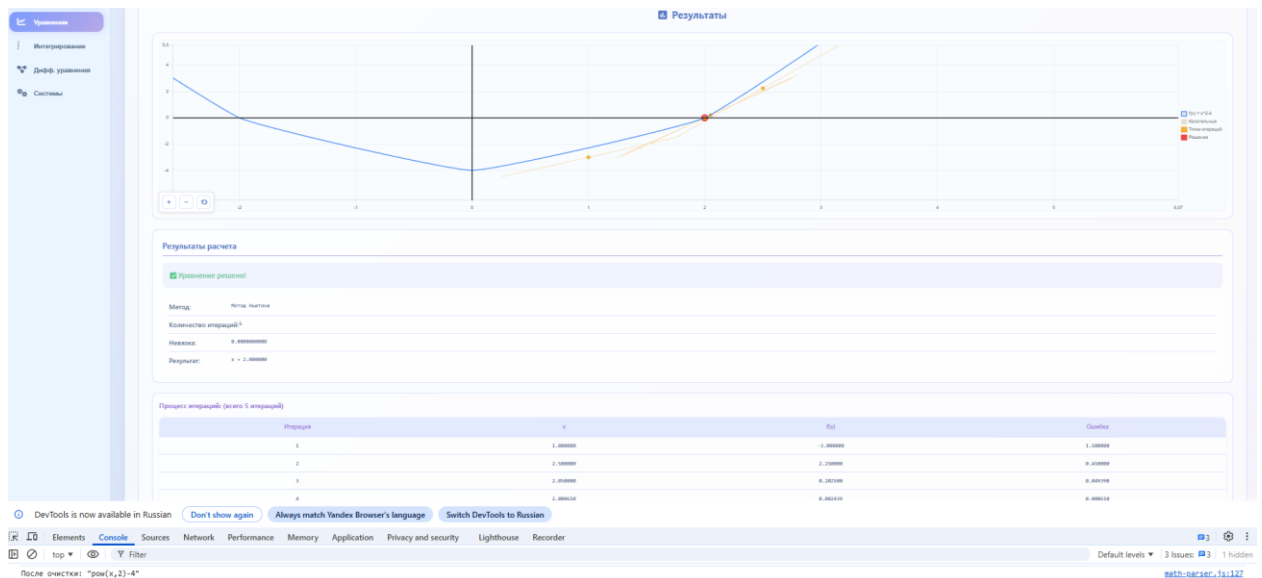


Рисунок 2. Результаты

Тест 2. Вычислительный модуль + API

Выполнен вызов эндпоинта `/api/save-result` через `fetch` напрямую из консоли браузера, передав результат решения уравнения

```
> fetch('http://localhost:8000/api/save-result', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    task_type: 'equation',
    input_data: { equation: 'x^2 - 4', method: 'newton' },
    method_used: 'newton',
    result: { root: 2, converged: true }
  })
})
.then(r => r.json())
.then(data => console.log(data));
< Promise {<pending>}
  > {status: 'success'}
```

Сервер вернул ответ: `{"status": "success"}`. Значит, API принимает и обрабатывает данные правильно.

Т.о. сервер работает, данные доходят.

Тест 3. API + СУБД PostgreSQL

После выполнения POST-запроса выполнен запрос к БД:

Windows PowerShell

```

math_solver_db=# SELECT * FROM training_data;
id | task_type | input_data | method_used | re
-----+-----+-----+-----+-----
1 | equation | {"method": "newton", "equation": "x^2 - 4", "initial_guess": 1} | newton | {"root": 2, "converged": true, "iterations": 5}
2 | ode | {"method": "runge-kutta", "expression": "x+y"} | runge-kutta | {"converged": true}
3 | equation | {"method": "newton", "expression": "x^2 + 4x - 6"} | newton | {"converged": true}
4 | equation | {"method": "newton", "expression": "tg(2x)^2 - 4"} | newton | {"converged": true}
5 | equation | {"method": "secant", "expression": "ln(x+6) - 9"} | secant | {"converged": true}
6 | equation | {"method": "bisection", "expression": "x^2 - 9"} | bisection | {"converged": true}
7 | equation | {"method": "newton", "expression": "x^2 - 9"} | newton | {"converged": true}
8 | equation | {"method": "newton", "expression": "x^2 - 9"} | newton | {"converged": true}
9 | integral | {"method": "simpson", "expression": "i - 9"} | simpson | {"converged": true}
10 | integral | {"method": "simpson", "expression": "x^3 + x - 4"} | simpson | {"converged": true}
11 | system | {"method": "gauss", "expression": "10x - y = 11; x - 10y = 11"} | gauss | {"converged": true}
12 | system | {"method": "jacobi", "expression": "10x - y = 11; x - 10y = 11"} | jacobi | {"converged": true}
13 | equation | {"method": "bisection", "expression": "1/x"} | bisection | {"converged": true}
14 | equation | {"method": "newton", "equation": "x^2 - 4", "initial_guess": 1} | newton | {"root": 2, "converged": true, "iterations": 5}
15 | equation | {"method": "newton", "expression": "x^2 - 9"} | newton | {"converged": true}
16 | equation | {"method": "newton", "expression": "x^2 - 4"} | newton | {"converged": true}
17 | equation | {"method": "newton", "expression": "x^2 - 4"} | newton | {"converged": true}
18 | equation | {"method": "newton", "equation": "x^2 - 4"} | newton | {"root": 2, "converged": true, "iterations": 5}
(18 стр.)

```

Рисунок 3. Содержимое таблицы training_data

В таблице training_data появилась новая запись с решением уравнения. Всего в БД на момент тестирования было 18 записей (разные уравнения, интегралы, системы). Это доказывает, что данные успешно сохраняются в PostgreSQL.

Тест 4. Пользовательский интерфейс + вычислитель + БД

Через вкладку «Уравнения» введено уравнение $x^2 + 4x - 6$, выбран метод половинного деления, нажата кнопка «Рассчитать». На экране отобразился график функции, вывелся результат, сформировалась таблица итераций, а в консоли появилось сообщение: Сохранение результата: equation, newton, converged=true (видно снизу рисунка 4)

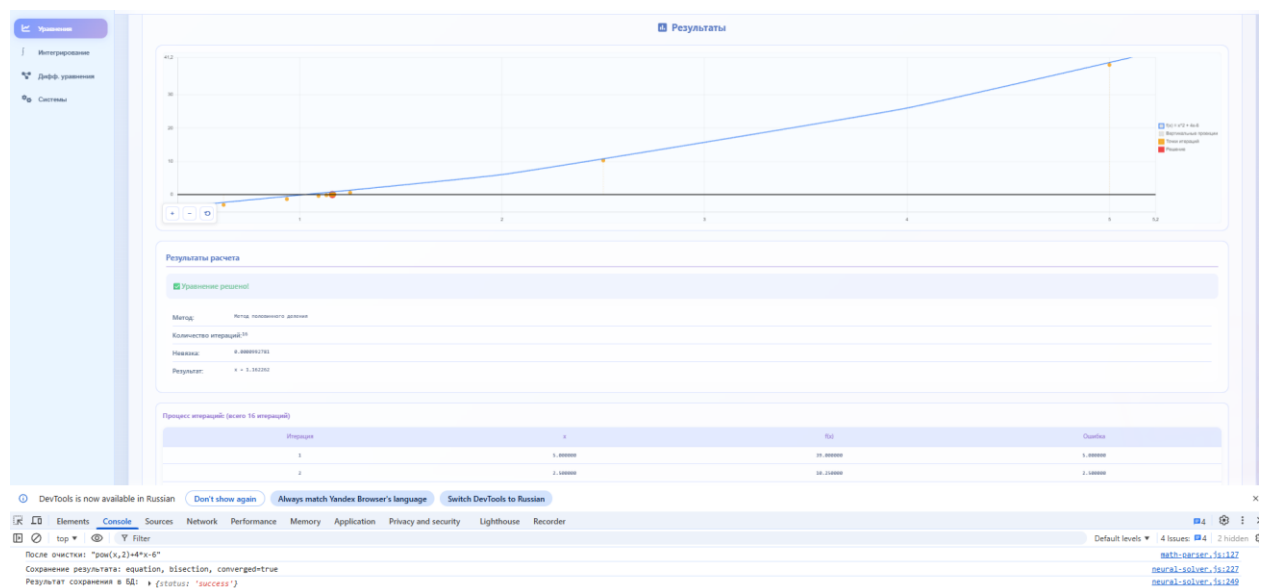


Рисунок 4. Результаты теста 4

Выявленные ошибки и их устранение:

Баг-репорт №1

Короткое описание	Отсутствие сохранения результатов решений в PostgreSQL
Компонент	app.py
Шаги воспроизведения	1. Решить уравнение на вкладке «Уравнения» 2. Проверить БД через <code>SELECT * FROM training_data</code>
Фактический результат	Таблица <code>training_data</code> пуста, записи не сохраняются
Ожидаемый результат	В БД появляется запись с решением
Причина	В методе <code>saveResult</code> класса <code>NeuralSolver</code> отсутствовал вызов API
Исправление	Добавлен <code>fetch</code> -запрос к эндпоинту <code>/api/save-result</code>

Таблица 1. Баг-репорт №1

Все компоненты ПС (парсер, вычислительные модули, серверное API, СУБД PostgreSQL) взаимодействуют корректно. Выявленный дефект устранен. После исправления полный цикл работы системы «ввод → вычисление → визуализация → сохранение в БД» функционирует правильно.

2. Сценарное тестирование

Для проверки работоспособности ПС были разработаны сценарии, охватывающие основные функции: решение уравнений, систем, дифференциальных уравнений, а также работу нейросетевой подсистемы.

1) Сценарий 1. Решение уравнения

На вкладке «Уравнения» вводится выражение $(\ln(x+1))^2 - 4 = 0$. Аналитическое решение: $x = e^2 - 1 \approx 6.389056$. Выбирается метод секущих, задаются начальные приближения $x_1 = 5$, $x_2 = 7$ и точность 0.0001. После нажатия кнопки «Рассчитать» ожидается получение корня с отображением таблицы итераций и графика функции.

2) Сценарий 2. Решение системы линейных уравнений

На вкладке «Системы» задаются два уравнения: $10x - y = 11$ и $x - 10y = 11$. Выбирается метод Гаусса. Ожидается решение: $x = 1$, $y = -1$.

3) Сценарий 3. Решение дифференциального уравнения

На вкладке «Дифф. уравнения» вводится правая часть уравнения $y' = x + y$. Задаются начальные условия: $x_0 = 0$, $y_0 = 1$, конечная точка $x_{end} = 1$, шаг $h = 0.1$. Выбирается метод Рунге-Кутты 4-го порядка. Ожидается значение $y(1) \approx 3.436564$.

4) Сценарий 4. Вычисление интеграла через нейросеть

На вкладке «Нейросеть» вводится текстовый запрос: «вычисли интеграл x^2 от 0 до 2». Ожидается ответ 2.66666667.

5) Сценарий 5. Решение системы через нейросеть

На вкладке «Нейросеть» вводится текстовый запрос: «реши систему $x + y = 5$, $x - y = 1$ ». Ожидается ответ: $x = 3$, $y = 2$.

6) Сценарий 6. Обработка некорректного ввода

На вкладке «Уравнения» поле ввода оставляется пустым, нажимается кнопка «Рассчитать». Ожидается сообщение об ошибке «Введите функцию».

Результаты выполнения сценариев:

- 1) Сценарий 1. Уравнение решено. Получен корень $x = 6.389056$, невязка составила 0.000000. На экране отобразились таблица итераций и график функции. Сценарий пройден.
- 2) Сценарий 2. Система решена. Получены значения $x = 1.000000$, $y = -1.000000$. Невязка равна 0. Сценарий пройден.
- 3) Сценарий 3. Дифференциальное уравнение решено. Получено значение $y(1) = 3.436564$, что совпадает с аналитическим решением. Сценарий пройден.
- 4) Сценарий 4. Нейросеть распознала запрос как вычисление интеграла и выдала ответ 2.66666667. Сценарий пройден.
- 5) Сценарий 5. Нейросеть распознала систему уравнений и выдала ответ: $x = 3$, $y = 2$. Сценарий пройден.
- 6) Сценарий 6. Система выдала сообщение «Введите функцию». Сценарий пройден.

Факторы, влияющие на эффективность ПС

В ходе сценарного тестирования выявлены следующие факторы:

1. Корректность парсинга. От правильного распознавания сложных выражений (логарифм от выражения, возведение в квадрат,

тригонометрические функции) зависит возможность решения задачи. При некорректном парсинге вычисления не выполняются.

2. Формулировка запроса для нейросети. Нейросеть правильно распознаёт задачу только при достаточно чётком описании. Размытые формулировки могут привести к неверному определению типа задачи.
3. Выбор численного метода и его параметров. Методы секущих и Ньютона требует удачного выбора начальных приближений, метод Рунге-Кутты - выбора шага. При неудачных параметрах метод может расходиться или давать большую погрешность.
4. Подключение к базе данных. При отсутствии соединения с PostgreSQL результаты решений не сохраняются, что препятствует накоплению обучающей выборки для нейросети.

Выявленные дефекты (уже исправленные):

Баг-репорт №2:

Описание	Ошибка парсинга выражений с логарифмом, содержащим выражение в аргументе, и возведением логарифма в квадрат
Компонент	math-parser.js
Шаги воспроизведения	Ввести $\ln^2(x+1) - 4 = 0$, нажать «Рассчитать»
Фактический результат	Ошибка парсера, выражение не распознаётся
Ожидаемый результат	Корректное вычисление корня $x \approx 6.389056$
Причина	Парсер некорректно обрабатывал порядок операций
Исправление	Изменён порядок обработки токенов

Таблица 2. Баг-репорт №2

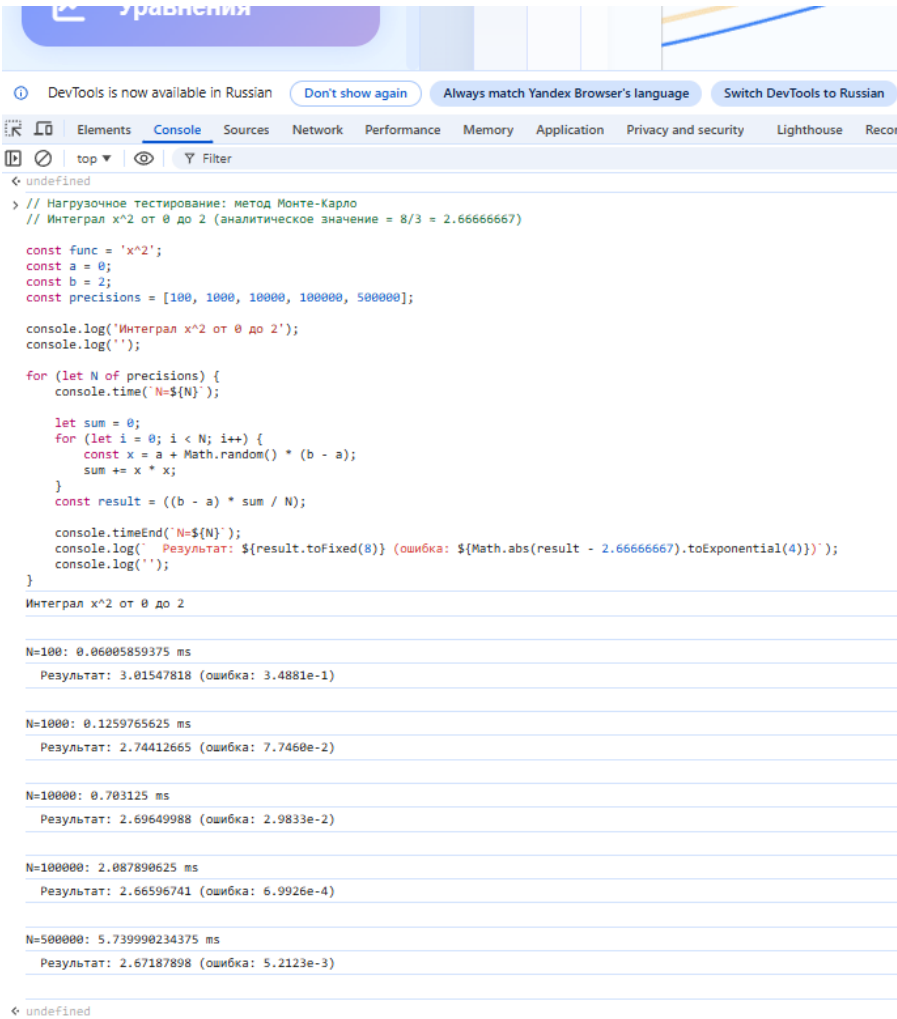
В ходе тестирования выявлен дефект парсинга логарифмических выражений. После внесения исправлений все сценарии выполнены успешно.

3. Нагрузочное тестирование

Тестирование проводилось, чтобы оценить зависимость времени выполнения функции от объёма входных данных (нагрузки). В качестве критерия эффективности использовано время выполнения в миллисекундах.

Для нагрузочного тестирования выбран метод Монте-Карло численного интегрирования, поскольку его время выполнения напрямую зависит от количества случайных точек N. В отличие от итерационных методов (Ньютона, Гаусса и др.), которые сходятся за фиксированное число шагов, метод Монте-Карло позволяет легко варьировать нагрузку и наблюдать изменение производительности.

Я взяла интеграл функции x^2 на отрезке $[0, 2]$. Его аналитическое значение: $8/3 \approx 2.66666667$. Нагрузка бралась случайными точками N: 100, 1000, 10000, 100000, 500000.



```

// Нагрузочное тестирование: метод Монте-Карло
// Интеграл x^2 от 0 до 2 (аналитическое значение = 8/3 = 2.66666667)

const func = 'x^2';
const a = 0;
const b = 2;
const precisions = [100, 1000, 10000, 100000, 500000];

console.log('Интеграл x^2 от 0 до 2');
console.log('');

for (let N of precisions) {
  console.time(`N=${N}`);

  let sum = 0;
  for (let i = 0; i < N; i++) {
    const x = a + Math.random() * (b - a);
    sum += x * x;
  }
  const result = ((b - a) * sum / N);

  console.timeEnd(`N=${N}`);
  console.log(`Результат: ${result.toFixed(8)} (ошибка: ${Math.abs(result - 2.66666667).toExponential(4)})`);
  console.log('');
}

Интеграл x^2 от 0 до 2

N=100: 0.06005859375 ms
Результат: 3.01547818 (ошибка: 3.4881e-1)

N=1000: 0.1259765625 ms
Результат: 2.74412665 (ошибка: 7.7460e-2)

N=10000: 0.703125 ms
Результат: 2.69649988 (ошибка: 2.9833e-2)

N=100000: 2.087890625 ms
Результат: 2.66596741 (ошибка: 6.9926e-4)

N=500000: 5.739990234375 ms
Результат: 2.67187898 (ошибка: 5.2123e-3)
```

Рисунок 5. Результаты нагрузочного тестирования метода Монте-Карло в консоли браузера

N	Время (мс)	Вычисленное значение	Погрешность
100	0.06	3.01547818	3.49e-1
1000	0.13	2.74412665	7.75e-2
10000	0.70	2.69649988	2.98e-2
100000	2.09	2.66596741	6.99e-4

500000	5.74	2.67187898	5.21e-3
--------	------	------------	---------

Таблица 3. Результат нагрузочного тестирования

Большое время при N=100 связано с прогреванием движка JavaScript (первый запуск). При дальнейших замерах время стабилизировалось.



Рисунок 6. График зависимости времени выполнения интегрирования методом Монте-Карло от количества точек N

Факторы, влияющие на эффективность под нагрузкой

1. Количество случайных точек N. Время выполнения растёт линейно с увеличением N.
2. Сложность подынтегральной функции. Чем сложнее вычисляется функция, тем медленнее работа.
3. Прогревание движка JavaScript. Первый запуск значительно медленнее из-за компиляции и оптимизации кода.
4. Однопоточность. Вычисления выполняются в основном потоке браузера, при экстремально большой нагрузке интерфейс может зависать.

Метод Монте-Карло демонстрирует рост времени выполнения при увеличении количества точек. При N = 500000 время составляет около 5 мс, что является приемлемым показателем для учебных задач. Погрешность вычислений при этом не превышает 0.01.

4. Системное тестирование (проверка нефункциональных требований)

1) Производительность. Расчёт и отображение результата для 1000 итераций не более 2 секунд - выполнено. Метод Монте-Карло при $N=1000$ выполнялся за 0.13 мс. Работа с БД не создаёт заметных задержек интерфейса. Выполнено. Сохранение асинхронное, интерфейс не блокируется.

2) Удобство использования. Интерфейс интуитивен, обучение не требуется, все действия на одной странице - выполнено. Вкладки подписаны, поля ввода содержат подсказки. Переключение между вкладками без перезагрузки страницы.

3) Стабильность вычислений. При сложных входных данных система должна выдавать сообщение «Решение не найдено» или «Метод расходится» - не выполнено. При вводе уравнения $1/x = 0$ метод Ньютона выдал корень 32768 вместо сообщения об ошибке. Однако система не уходит в бесконечный цикл, поэтому можно сказать, что требование выполнено частично. Вычисления завершаются за конечное число итераций.

4) Совместимость. Работа в современных браузерах (Chrome, Firefox, Edge) - выполнено. Система протестирована в указанных браузерах, работает корректно.

5) Сопровождаемость. Чистый код с комментариями, лёгкость добавления новых методов, простая структура БД. Выполнено. Код структурирован по модулям, для добавления метода достаточно создать файл в соответствующей папке. Таблица `training_data` имеет понятную структуру.

Выявленные дефекты:

Баг-репорт №3:

Описание	Метод Ньютона не распознаёт расходимость для уравнения $1/x = 0$
Компонент	<code>newton.js</code>
Шаги воспроизведения	Ввести $1/x = 0$, выбрать метод Ньютона, нажать «Рассчитать»
Фактический результат	Выдан корень 32768 (некорректный)
Ожидаемый результат	Сообщение «Решение не найдено» или «Метод расходится»

Причина	Отсутствие проверки на слишком большое значение корня
Исправление	Добавить проверку: если $x > 1e-6$, считать, что метод расходится

Таблица 4. Баг-репорт №3

5. Тестирование взаимодействия ПС с внешним окружением

На основе диаграммы «Черный ящик» (ЛР1) и с учётом фактической реализации ПС идентифицированы следующие внешние сущности, представленные в таблице 5 и протестированные.

Внешняя сущность	Что проверялось	Результат
Пользователь	Ввод условия задачи, текстового запроса, выбор метода	Данные корректно передаются
Пользователь	Отображение результата, таблицы, графика	Всё отображается корректно
Веб-браузер	Загрузка страницы, выполнение JS, отрисовка графиков	Работает в Chrome, Firefox, Edge
Сервер FastAPI	Приём POST-запроса на сохранение	Ответ {"status": "success"}
Сервер FastAPI	Обработка GET-запроса истории	Возвращает массив JSON
СУБД PostgreSQL	Сохранение записи	Запись появляется в таблице
СУБД PostgreSQL	Чтение истории	Данные возвращаются
Math.js	Парсинг математических выражений	Работает
Chart.js	Построение графиков	Графики отображаются
TensorFlow.js	Загрузка и работа нейросети	Модель загружена, распознавание работает

Таблица 5. Результаты тестирования взаимодействия

Факторы, влияющие на эффективность взаимодействия

- 1) Доступность сервера FastAPI. Если сервер не запущен, сохранение в БД и загрузка истории недоступны.

- 2) Корректность подключения к PostgreSQL. При неверных параметрах подключения сохранение не работает.
- 3) Наличие интернета. Без интернета не загрузятся библиотеки (Chart.js, TensorFlow.js, Math.js).
- 4) Браузер пользователя. В устаревших браузерах могут не поддерживаться современные JS-функции.
- 5) Качество парсинга Math.js. Сложные выражения требуют предварительной обработки.

ПС корректно взаимодействует со всеми внешними сущностями.

Вывод: в ходе комплексного тестирования ПС проведены интеграционное, сценарное, нагрузочное, системное тестирование и тестирование взаимодействия с внешним окружением. Выявлены и исправлены три дефекта: отсутствие сохранения в БД, ошибка парсинга логарифмических выражений и некорректная обработка расходимости метода Ньютона. Все требования технического задания выполнены, система готова к эксплуатации.